

In-Depth Analysis of Homomorphic Encryption Libraries

Itai Zloclzower Jacob Shemesh Tomer Ovadia Yoni Glickstein

Confidential Computing Course
Ben-Gurion University of the Negev
July 2026

Abstract

Homomorphic encryption lets a server compute on data it cannot read, which closes the gap left open while data is being processed. The price is performance, and the question this project asks is how large that price actually is. We implement the same set of computations three ways — in plaintext, in TenSEAL, and in OpenFHE — and measure runtime, memory, ciphertext size and accuracy for each. Averaged over 1000 repetitions per operation, encrypted addition costs 153 times a plaintext addition in TenSEAL, while an encrypted summation over the same vector costs 32,832 times its plaintext equivalent: the overhead is not one number but a range spanning two orders of magnitude, and its size is governed by whether an operation needs ciphertext rotations. Two further experiments explain most of what we measured. Growing the input from 5 to 500 values increases encrypted runtime by only $7.3\times$ while plaintext runtime grows $33.8\times$, because CKKS packs many values into one ciphertext and charges by the ciphertext. And when OpenFHE is re-run at TenSEAL’s parameters rather than its own defaults, its multiplication penalty falls from $6.89\times$ to $3.16\times$, which means much of the apparent difference between the libraries is a difference in configuration rather than in code. Applying all of this to a synthetic healthcare score over 1000 records, the encrypted computation completes in 214 ms and agrees with the plaintext result to five significant figures.

1 Introduction

Encryption protects data at rest and in transit, but not while it is being used. To compute on encrypted data, a conventional system must first decrypt it, and at that moment the plaintext is exposed to whatever machine is doing the work. Homomorphic encryption (HE) removes that step: operations performed on ciphertexts produce ciphertexts that decrypt to the correct result, so a cloud provider can process data it never sees.

The idea is old and the implementations are recent. Several mature libraries now exist, which turns an abstract capability into a practical engineering decision: if a hospital or a bank wanted to outsource a computation over sensitive records today, what would it cost, and which library should it choose? Answering that requires measurement rather than theory, because the published asymptotics say little about what happens when a specific library runs a specific operation on a specific machine.

This project measures it. We compare two CKKS implementations, **TenSEAL** and **OpenFHE**, against an unencrypted baseline over a small set of computations common in data analysis, and report runtime, memory, ciphertext size, accuracy, and the practical difficulties we met while building them.

Research question. *What is the practical overhead of performing basic computations using two different homomorphic encryption implementations, compared to equivalent plaintext computation?*

The report follows the measurements in the order they built on one another. Section 2 covers the scheme and the libraries. Section 3 describes what we implemented and Section 4 how we

measured it. Section 5 gives the core result: the cost of each operation, and why it varies so widely. Sections 6 and 7 then test two explanations for that variation — input size and parameter choice. Section 8 applies the whole thing to a healthcare example, and Sections 9–10 report usability and the limitations we hit.

2 Background

2.1 Homomorphic encryption and CKKS

Both libraries here implement **CKKS** [1], a scheme for approximate arithmetic on real numbers. Approximate is the key word: decrypting a CKKS result returns the correct value plus a small error, in exchange for far better performance than exact schemes on real-valued data. For statistics over measurements this trade is usually acceptable, and Section 5.4 quantifies the error we observed.

Four properties of CKKS shape every measurement in this report.

- **Ring dimension.** Ciphertexts are polynomials. The degree of that polynomial, the ring dimension, sets the security level and determines the cost of every operation. A larger ring is more secure and uniformly slower.
- **Slot packing.** One ciphertext holds many values in parallel “slots”, and one homomorphic operation acts on all of them at once. Cost depends on the ring dimension, not on how many slots actually hold data — Section 6 shows what this implies.
- **Multiplicative depth.** Each multiplication consumes a level from a finite budget fixed at setup. Exceeding it does not slow the computation down; it makes the result wrong.
- **Rotations.** Slots cannot be addressed individually. Summing across them requires rotating the ciphertext and adding repeatedly, roughly $\log_2(\text{slots})$ times, each rotation involving an expensive key switch. This is why summation costs far more than addition, as Section 5.1 shows.

2.2 The two libraries

TenSEAL is a Python library built on Microsoft SEAL. It presents encrypted vectors as ordinary Python objects supporting `+`, `*` and `.sum()`, and manages relinearization and rescaling internally. The programmer supplies the modulus chain directly.

OpenFHE is a C++ library with Python bindings, maintained by an academic consortium. It exposes lower-level calls (`EvalAdd`, `EvalMult`, `EvalSum`) and requires evaluation keys to be generated explicitly. The programmer supplies a multiplicative depth and a scaling size, and the library derives the ring dimension needed to satisfy them at a chosen security level.

That difference in configuration style matters more than it first appears. Given what we consider comparable settings, the two libraries end up at different ring dimensions — 8192 for TenSEAL, 16384 for OpenFHE — which we return to in Section 7.

2.3 Related work

Comparing HE libraries is not a new exercise. Sathya et al. [4] survey the available libraries and their trade-offs. Faneela et al. [5] benchmark SEAL against OpenFHE across BGV and CKKS on two operating systems and conclude that OpenFHE is the stronger performer, and Takeshita et al. [6] profile four CKKS implementations while explicitly addressing the difficulty of comparing libraries whose architectures differ. Viand et al. [7] survey the compiler layer above these libraries.

Our headline ordering differs from Faneela et al., which is one reason Section 7 exists: we found that the ranking depends on the parameters each library is given, so a comparison at default settings measures the defaults as much as the code. We claim no methodological novelty over the works above; this is a course project, and its aim is a clear practical measurement of two libraries we implemented ourselves.

3 Implementation

We implemented the same seven operations three times: once in plain Python as a baseline, once in TenSEAL, and once in OpenFHE.

Operation	What it does
Key generation	Build the crypto context and generate the keys each library needs
Encryption	Encode a vector of numbers into a ciphertext
Addition	Element-wise sum of two encrypted vectors
Multiplication	Element-wise product of two encrypted vectors
Summation	Sum of all elements within one encrypted vector
Average	Summation followed by multiplication by $1/n$
Dot product	Element-wise multiplication followed by summation
Decryption	Recover the plaintext result

Table 1: The implemented operations. All except key generation and encryption have a direct plaintext counterpart used as the baseline.

The two libraries need different code for the same computation. Encrypted addition in TenSEAL is `enc + enc`; in OpenFHE it is `cc.EvalAdd(ct, ct)`. Summation is `enc.sum()` against `cc.EvalSum(ct, n)`, and the latter requires summation keys to have been generated beforehand.

One incompatibility appeared immediately and affected everything after it: **OpenFHE requires the batch size to be a power of two**, while TenSEAL accepts any vector length. Every input therefore has to be padded up to the next power of two. For simple operations this is harmless, but Section 8 shows a case where the padding value itself has to be chosen carefully.

Beyond the required operations we implemented three extensions: the encrypted dot product listed above, a sweep over input sizes (Section 6), and a re-run of OpenFHE under TenSEAL’s parameters (Section 7).

4 Experimental Setup

4.1 What is measured, and how

Runtime. Each operation is timed on its own with `time.perf_counter` around a single call. Encryption and decryption are timed separately from the operations between them, so the cost of getting data in and out is never folded into the cost of computing on it.

Repetitions and warm-up. A single timing of an HE operation is not reproducible. Every figure in this report is the **mean of 1000 timed repetitions**, preceded by **50 warm-up calls that are discarded**. The warm-up matters because the first calls into either library pay costs that are not part of steady-state operation: loading the native library, building lookup tables on first use, and faulting in freshly allocated memory. Including them would inflate the mean and, worse, make it depend on how many repetitions were run. Garbage collection is disabled inside the timed loop, so an unrelated collection is not charged to the operation being measured. Every repetition starts from the same freshly encrypted inputs, so no repetition inherits accumulated noise or a consumed level from the one before it.

Memory. We report two figures because one is not enough. Python’s `tracemalloc` sees only Python-level allocations; both libraries keep ciphertext coefficients in native C++ memory that Python never learns about, so `tracemalloc` alone reports the size of a wrapper object rather than of a ciphertext. We therefore also allocate many ciphertexts, hold them, and measure the growth in the process resident set. Memory is measured in a separate pass from timing, because `tracemalloc` adds a hook to every allocation and distorts short measurements.

Ciphertext size. Each library’s own serialization routine, measured in bytes, compared against the plaintext vector as 8-byte doubles.

Accuracy. Every encrypted result is compared against the plaintext computation of the same

values, and the maximum absolute error is reported. Each benchmark asserts this error is within a loose bound before reporting any timing, so a run whose parameters were wrong fails visibly instead of producing plausible but meaningless numbers.

4.2 Machine and configuration

Measurements ran on a compute node reserved exclusively for the job, so no other work shared the machine. We fixed the thread count to one (OMP_NUM_THREADS=1): both libraries use multi-threaded native code, and on a shared machine their measured cost would depend on how many cores each happened to obtain, which is not a property of the library.

CPU	Intel Xeon E5-2680 v3 @ 2.50 GHz, single-threaded, exclusive node
OS / Python	Rocky Linux 9.7 / Python 3.8.20
Libraries	TenSEAL 0.3.15, OpenFHE 1.4.1, NumPy 1.24.4
TenSEAL parameters	poly_modulus_degree = 8192, coefficient moduli [60, 40, 40, 60], scale 2^{40}
OpenFHE parameters	multiplicative depth 3, scaling modulus 50 bits, 128-bit security $\Rightarrow$ ring 16384
Input	The vector [1.0, 2.0, 3.0, 4.0, 5.0] unless stated otherwise

Table 2: Experimental configuration. The two parameter sets are each library’s natural way of expressing comparable settings; Section 7 examines what that difference costs.

Because absolute timings depend on this hardware, the ratios between the three columns are the transferable result rather than the millisecond values themselves.

5 Results: The Cost of Basic Operations

5.1 Runtime

Operation	Plaintext (ms)	TenSEAL (ms)	OpenFHE (ms)	TenSEAL overhead	OpenFHE overhead
Encrypt	—	5.4914	15.1101	—	—
Add	0.000629	0.0961	0.3869	153×	615×
Multiply	0.000550	4.7413	32.4827	8,616×	59,025×
Sum	0.000320	10.5082	76.1546	32,832×	237,943×
Average	0.000349	11.7961	81.3563	33,794×	233,072×
Dot product	0.000993	11.0764	96.6155	11,152×	97,272×
Decrypt	—	1.4029	22.6702	—	—
Total (5 comparable ops)	0.002841	38.2180	286.9961	13,450×	101,003×

Table 3: Mean runtime over 1000 repetitions. The total covers the five operations that exist in all three columns; encryption and decryption have no plaintext counterpart and are excluded from it.

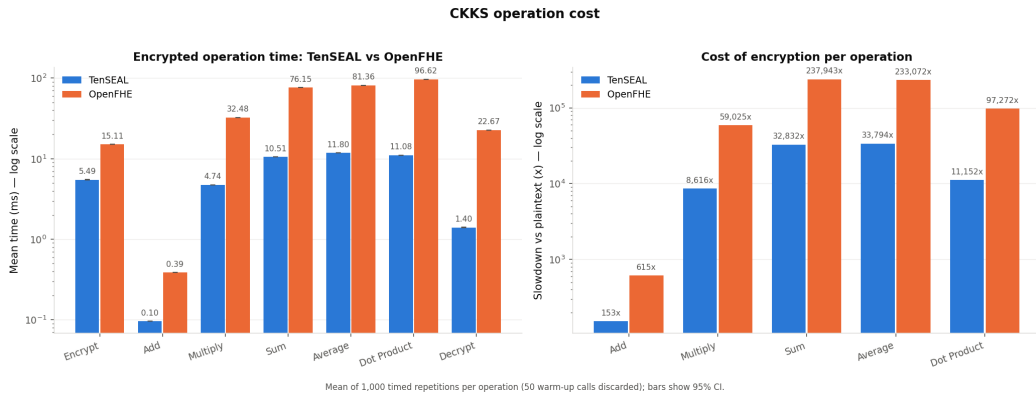


Figure 1: Runtime per operation on a logarithmic scale, and the slowdown against plaintext. Both libraries span two orders of magnitude across operations.

Three things stand out.

There is no single overhead figure. Within TenSEAL alone the cost ranges from $153\times$ for addition to $32,832\times$ for summation. A statement such as “HE is ten thousand times slower” is therefore not usable: the answer depends entirely on which operation dominates the workload.

Addition is nearly free; rotation-heavy operations are not. Encrypted addition takes 96 microseconds, a hundredth of the cost of a multiplication and a hundredth of a summation. The reason is structural. Addition acts on ciphertext coefficients directly. Summation, average and dot product must move data between slots, and each rotation carries a key switch. This single mechanism explains the ordering of every row in the table, and it is the most actionable thing we measured: a computation that can be expressed with additions and constant multiplications will be far cheaper than one that needs to aggregate across slots.

TenSEAL is faster here, on every operation. It leads by $2.75\times$ on encryption and by $6.89\times$ on multiplication. Section 7 tests whether that reflects the libraries or the parameters we gave them.

5.2 Key generation

Key generation happens once per session rather than once per operation, so we timed it separately, over 50 repetitions.

	TenSEAL	OpenFHE
Context and key generation	216.180 ms	208.308 ms

This is the one measurement where OpenFHE is ahead, and the two are within 4% of each other. The more useful observation is the scale: TenSEAL spends 216 ms building keys and 38 ms performing all five encrypted operations. Setup costs almost six times the computation, so a service that builds a fresh context for every request would spend most of its time on key generation rather than on the work it was asked to do.

5.3 Memory and ciphertext size

Measure	TenSEAL	OpenFHE
Python-visible memory (<code>tracemalloc</code>), encryption	0.59 KB	0.42 KB
Resident memory held per ciphertext	359,465 B	1,258,455 B
Serialized ciphertext	334,428 B	1,312,467 B
Plaintext vector (5 doubles)	40 B	

Table 4: Memory for the 5-element vector. The first row measures Python objects; the second and third measure the ciphertext itself.

The first row is the reason we measured memory two ways. Taken alone, `tracemalloc` suggests a ciphertext costs under a kilobyte, which is wrong by a factor of roughly 600: it is measuring the Python wrapper, not the encrypted data behind it. The resident-set figure and the serialized size agree with each other to within 7%, which is reassuring since they are independent measurements of the same object.

The practical consequence is the last row. Five numbers occupy 40 bytes in the clear and 334 KB encrypted under TenSEAL, an expansion of roughly $8,400\times$. For a cloud deployment this, rather than RAM, is the binding constraint, because it is what has to cross the network. Section 8 shows the ratio improving sharply once the ciphertext slots are actually used.

We also measured the evaluation keys, which the server needs before it can compute at all. At the configuration used in Section 8 these are **179.7 MB** for TenSEAL and **69.2 MB** for OpenFHE, against roughly 5 MB of encrypted data. They are transferred once per context rather than per query, but they are by a wide margin the largest object involved.

5.4 Accuracy

Operation	TenSEAL	OpenFHE
Add	1.57×10^{-9}	4.09×10^{-14}
Multiply	3.35×10^{-6}	1.96×10^{-12}
Sum	4.60×10^{-7}	7.11×10^{-15}
Average	3.10×10^{-7}	5.20×10^{-14}
Dot product	5.19×10^{-6}	3.27×10^{-13}

Table 5: Maximum absolute error against the plaintext result.

Both libraries are accurate enough for the applications CKKS targets: an error of 10^{-6} on values of order 10 is far below the precision of any measurement one would realistically encrypt. Multiplication produces the largest error in both, which follows from the scheme — multiplying two ciphertexts multiplies their noise, and the rescaling that follows discards low-order bits.

OpenFHE is several orders of magnitude more precise throughout. Part of that is its larger scaling factor, 2^{50} against TenSEAL’s 2^{40} , which retains about ten more bits of each value. Section 7 shows that this is not the whole explanation.

6 Extension: Larger Inputs

The operations above ran on five values, which is small enough that the question arises whether the overheads say anything about realistic workloads. We repeated the full pipeline — encrypt, add, multiply, sum, average, decrypt — at input sizes from 5 to 500.

Input size	Plaintext (ms)	TenSEAL (ms)	OpenFHE (ms)	TenSEAL ovh.	OpenFHE ovh.
5	0.001309	37.16	301.55	$28,389\times$	$230,368\times$
10	0.001683	51.13	347.45	$30,383\times$	$206,446\times$
50	0.004772	98.95	451.32	$20,735\times$	$94,576\times$
100	0.008711	119.99	501.16	$13,774\times$	$57,532\times$
500	0.044284	271.19	607.03	$6,124\times$	$13,708\times$

Table 6: End-to-end pipeline time by input size. Key generation happens once per size and is excluded.

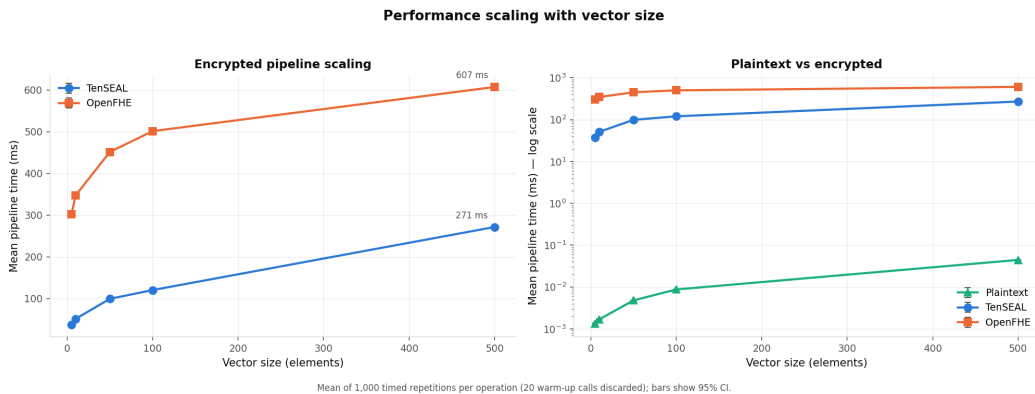


Figure 2: Pipeline time against input size. Left: the two encrypted libraries. Right: all three on a logarithmic scale.

Across a hundredfold increase in data, plaintext time grows $33.8\times$ — close to linear, as expected — while TenSEAL grows $7.3\times$ and OpenFHE only $2.0\times$. The cost per element falls accordingly, from 7.43ms to 0.54ms for TenSEAL.

This is slot packing at work. The cost of a CKKS operation is fixed by the ring dimension, not by how many slots contain data, so processing 500 values costs little more than processing 5. What does grow is the rotation count in the summation, which rises with $\log_2$ of the batch size.

The practical reading is that **a partly empty ciphertext is wasted work**. At five values, TenSEAL is paying for thousands of slots to compute on five numbers, which is why the overhead there is $28,389\times$ and at 500 values only $6,124\times$. Neither library is full even at 500. It also means an overhead figure quoted without the input size behind it says very little: the same pipeline in this one table ranges from $230,368\times$ to $13,708\times$ for OpenFHE.

7 Extension: Are the Libraries Different, or the Parameters?

Section 5.1 found TenSEAL faster on every operation and Section 5.4 found OpenFHE more precise. Both comparisons have the same weakness: the two libraries were not running equivalent parameters. OpenFHE had chosen ring dimension 16384 against TenSEAL’s 8192, so it was doing roughly twice the polynomial work per operation, and it was using a 2^{50} scaling factor against 2^{40} , so it was keeping ten more bits of every value. Attributing either result to the library rather than to the configuration would be unfounded.

OpenFHE allows the ring dimension to be set explicitly, so we re-ran it pinned to 8192 with a 2^{40} scale, matching TenSEAL. We also varied OpenFHE’s *scaling technique*, an internal option controlling how it manages the scaling factor between multiplications, which TenSEAL does not expose.

OpenFHE configuration	Encr	Add	Mult	Sum	Dot	Decr	Geo. mean
Own defaults (ring 16384, 2^{50})	2.52	4.02	6.89	7.24	8.70	16.28	6.44
Matched, same scaling technique	1.20	2.12	3.16	3.25	3.97	7.83	3.05
Matched, fastest scaling technique	1.02	1.77	1.93	2.41	3.09	6.80	2.37

Table 7: OpenFHE runtime as a multiple of TenSEAL’s, per operation. Lower is closer to TenSEAL.

Matching the parameters closes most of the gap. Multiplication falls from $6.89\times$ to $3.16\times$ and encryption becomes almost identical at $1.20\times$; across all six operations the geometric mean falls from $6.44\times$ to $3.05\times$. Separating the two changes, the ring dimension accounts for a factor of 2.18 and the scaling technique for a further 1.64.

Two qualifications belong with this result. Decryption remains $7.8\times$ slower even matched, so the gap narrows everywhere but closes nowhere. And pinning the ring dimension overrides OpenFHE’s own security check: asked to select a ring for these parameters at a 128-bit security level, OpenFHE chooses 16384 and rejects 8192. The matched configuration therefore buys comparability at the cost of security, which is acceptable for an experiment isolating one variable but would not be acceptable in deployment.

The conclusion we draw is that **a benchmark of two HE libraries at their respective defaults measures the defaults as much as the libraries**. This also offers an explanation for why Faneela et al. [5] rank OpenFHE ahead of SEAL where we rank TenSEAL ahead of OpenFHE: at default settings the two are not performing equivalent work.

8 Use Case: Privacy-Preserving Healthcare Analytics

The measurements so far concern isolated operations. A real workload composes them, and composing them raises questions that single operations do not. To see this we implemented the motivating scenario from our proposal: a hospital wants a cloud provider to compute a score for every patient and the average over the cohort, without exposing any individual record.

The score. We define a **synthetic risk score** over five measurements per patient. *This score is constructed for demonstration only. It is not a medical model and carries no clinical meaning.* Each feature is first normalised to roughly $[0, 1]$ against a fixed public range, $x_{\text{norm}} = (x - \min)/(\max - \min)$, using age 18–90, systolic blood pressure 90–180, BMI 18–40, cholesterol

120–280 and glucose 70–200. The score is then

$$\text{score} = 100 \cdot (0.15a + 0.20b + 0.15m + 0.15c + 0.15g + 0.10bg + 0.05mc + 0.05b^2),$$

where a, b, m, c, g are the normalised age, blood pressure, BMI, cholesterol and glucose. The eight weights sum to 1, so a patient at the top of every range scores 100.

This form was chosen to exercise the library rather than to model medicine. The first five terms are a weighted average, needing only multiplication by public constants. The last three are what make it a useful test: two interaction terms and one square, each requiring the multiplication of one ciphertext by another. That is the operation that consumes multiplicative depth, and it is the reason this computation cannot run under the parameters used in Section 5.

The data. A synthetic cohort of 1000 patients, generated with a fixed seed. Age is drawn uniformly across its range; the four clinical measures are drawn from normal distributions centred on plausible values and then clipped to the ranges above, which keeps every normalised value inside $[0, 1]$. The resulting scores have mean 35.63 and standard deviation 8.45.

Encrypting it. We encrypt one *feature* per ciphertext, with one patient per slot, giving five ciphertexts of 1000 values each. A single homomorphic multiplication then advances all 1000 patients at once. The alternative — one ciphertext per patient — would multiply the number of operations by a thousand.

The padding rule from Section 3 becomes delicate here. Vectors are padded from 1000 to 1024 slots, and we fill the padding with *each feature’s minimum*, not with zero. A minimum normalises to exactly 0, so padded slots contribute nothing to any term and drop out of the cohort sum. Zero-padding would instead give those slots a normalised value of $-\min/(\max - \min)$, which is not zero, and would silently corrupt the average. This is exactly the kind of error that produces a plausible wrong answer, so we cover it with a unit test.

Depth. Normalisation, the interaction terms, the weights and the cohort mean consume four levels in total. Neither library can run this at the Section 5 settings: TenSEAL fails with **scale out of bounds** and a longer modulus chain does not fit at ring 8192, so the ring must double to 16384; OpenFHE evaluates the score at depth 3 but fails on the cohort mean, and needs depth 4. Adding two interaction terms and a square to a weighted average was enough to force a change of cryptographic parameters and a doubling of the ring dimension — a useful illustration that in CKKS the shape of a computation matters more than the amount of data in it.

8.1 Results

Stage	Runs at	Plaintext (ms)	TenSEAL (ms)	OpenFHE (ms)
Encrypt 5 features	Hospital	—	76.784	84.650
Evaluate the score	Cloud	—	89.249	163.181
Cohort mean	Cloud	—	46.104	146.054
Decrypt the scores	Hospital	—	2.186	17.457
Total		0.0748	214.323	411.341
Overhead		1×	2,865×	5,499×
Per patient		0.000075	0.2143	0.4113

Table 8: The full scenario over 1000 patients. The plaintext column is the single computation producing the same output: a score per patient and the cohort mean.

	TenSEAL	OpenFHE
Maximum absolute error (score points)	4.881×10^{-4}	1.976×10^{-10}
Cohort mean, encrypted	35.627391	35.626948
Cohort mean, plaintext	35.626948	

Table 9: Accuracy of the encrypted score. The worst per-patient error is 5×10^{-4} points on a 0–100 scale.

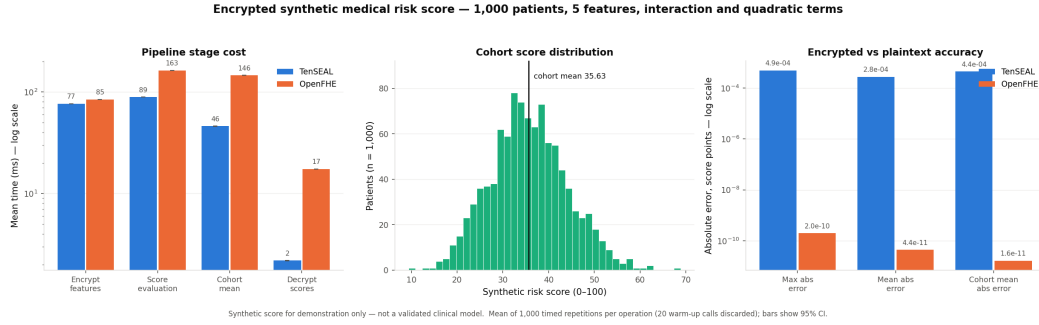


Figure 3: Cost per stage, the distribution of scores across the synthetic cohort, and the accuracy of the encrypted result.

The whole scenario takes **214 ms** with TenSEAL and 411 ms with OpenFHE, of which the cloud performs 135 ms and 309 ms. Per patient that is 0.21 and 0.41 ms, and the encrypted cohort mean matches the plaintext value to five significant figures.

The overhead is worth comparing against Section 5: 2,865× here, against 13,450× for the isolated operations. Nothing became faster. The ciphertexts became fuller — 1024 slots carrying real data instead of 5 — which is the same effect Section 6 isolated, now visible in a complete workload. Measuring HE on toy inputs overstates its cost.

The same amortisation appears in the data volume. Each encrypted feature is about 1 MB, so the hospital uploads roughly 5 MB for a cohort that occupies 40 KB in the clear — an expansion of 132×, against the 8,400× measured on the 5-element vector in Section 5. The evaluation keys, at 179.7 MB, remain larger than the data.

Two honest caveats. Encryption and decryption are performed by the hospital, so 79 ms of the 214 ms is work the client does rather than work it avoids; for a computation this small, outsourcing would not pay for itself, and the scenario is better read as a stand-in for workloads where it would. And our benchmark runs client and server in a single process, so it measures the computation faithfully but not the network transfer or key management a deployment would also need.

9 Ease of Implementation and Usability

Aspect	TenSEAL	OpenFHE
Installation	<code>pip install</code> , any platform	Linux only, one Python version
Writing a computation	Python operators on encrypted vectors	Explicit method calls
Key management	Handled by the context	Evaluation keys generated by hand
Parameters	Modulus chain given directly	Depth given, ring dimension derived
Vector length	Any	Powers of two only
Error messages	Python exceptions	C++ errors through the bindings
Benchmark code	92 lines	111 lines

TenSEAL was considerably easier to work with. Encrypted code looks like the plaintext code beside it, and the context handles keys without being asked. OpenFHE requires more from the programmer: evaluation keys must be generated explicitly, batch sizes rounded to powers of two, and several concepts named that TenSEAL keeps internal.

That verbosity is not purely a cost. OpenFHE requires the multiplicative depth to be declared up front, so exceeding it fails immediately and says so. TenSEAL discovers the same problem later and reports **scale out of bounds**, whose connection to the length of the modulus chain is not obvious to a newcomer — we spent noticeably longer diagnosing that failure in TenSEAL than the equivalent in OpenFHE. OpenFHE also exposes controls, such as the scaling technique of Section 7, that turn out to matter and that TenSEAL does not offer at all.

10 Practical Limitations

Encountered during development.

1. **OpenFHE installs only on Linux, and only for one Python version.** Its wheel ships a binary built for CPython 3.8; installing it under 3.9 or later appears to succeed and then fails at import. This dictated the Python version for the whole project.
2. **Power-of-two batch sizes.** Every input must be padded, and Section 8 shows the padding value is not always free to choose.
3. **Multiplicative depth must be planned in advance.** Adding two interaction terms to a weighted average forced new parameters and doubled the ring dimension. Asking for more depth than needed is equally costly: our first attempt requested depth 5, which works but pushes the ring to 32768 and slows every operation for no benefit.
4. **Measuring memory is not straightforward.** The obvious tool reports a figure roughly 600 times too small, because the data lives outside Python.

Limitations of this study.

1. Measurements come from one machine, one CPU generation and a single thread. The ratios between columns transfer better than the absolute times.
2. The plaintext baseline is ordinary Python. A NumPy or C baseline would be faster, so our overhead figures are upper bounds relative to optimised plaintext.
3. Each configuration was measured in one process run, so run-to-run variation between processes is not captured.
4. Neither library was tested with bootstrapping, so all results concern computations that fit within a fixed level budget.

11 Conclusions

The research question asked what basic computations cost under homomorphic encryption compared with plaintext. The measurements give a specific answer and, more usefully, explain what governs it.

1. **The overhead is a range, not a number.** Encrypted operations cost between $153\times$ and $32,832\times$ their plaintext equivalents in TenSEAL, and between $615\times$ and $237,943\times$ in OpenFHE. Any single quoted figure hides a factor of two hundred.
2. **What an operation costs depends on whether it needs rotations.** Addition and constant multiplication act on ciphertexts directly and are cheap. Summation, average and dot product must move data between slots, and each rotation carries a key switch. Structuring a computation to avoid aggregation across slots is the single most effective optimisation available.
3. **Ciphertexts should be filled.** A hundredfold increase in input raised encrypted runtime by only $7.3\times$, because CKKS charges per ciphertext rather than per value. The corollary is that measurements on small inputs are misleading: the same pipeline showed $28,389\times$ overhead at five values and $6,124\times$ at five hundred.
4. **Much of the difference between the libraries was a difference in parameters.** Re-running OpenFHE at TenSEAL’s ring dimension and scale cut its multiplication penalty from $6.89\times$ to $3.16\times$ and made encryption nearly identical. A comparison of two HE libraries at their default settings is partly a comparison of the defaults.
5. **Composite computations are cheaper than isolated operations suggest.** The health-

care scenario, over 1000 records with interaction terms and a cohort aggregate, ran at $2,865\times$ plaintext — a fifth of the overhead of the isolated benchmark — and agreed with the plaintext result to five significant figures.

- 6. Data volume is dominated by keys, not ciphertexts.** A ciphertext expands the data $132\times$ at full occupancy, but the evaluation keys the server needs are 179.7MB against 5MB of encrypted data.

Overall, homomorphic encryption is usable today for the kind of computation examined here, provided the workload is batched, structured to avoid unnecessary rotations, and given parameters chosen deliberately rather than by default. The cost is real but is dominated by choices the programmer controls.

A further experiment beyond the scope of this proposal, measuring the cost of OpenFHE’s noise-flooding decryption mode, is included in the project repository.

References

- [1] J. H. Cheon, A. Kim, M. Kim, Y. Song. Homomorphic encryption for arithmetic of approximate numbers. *ASIACRYPT*, 2017.
- [2] A. Al Badawi et al. OpenFHE: open-source fully homomorphic encryption library. *WAHC*, 2022.
- [3] A. Benaïssa, B. Retiat, B. Cebere, A. E. Belfedhal. TenSEAL: a library for encrypted tensor operations using homomorphic encryption. 2021.
- [4] S. S. Sathya, P. Vepakomma, R. Raskar, R. Ramachandra, S. Bhattacharya. A review of homomorphic encryption libraries for secure computation. arXiv:1812.02428, 2018.
- [5] Faneela, J. Ahmad, B. Ghaleb, S. U. Jan, W. J. Buchanan. Cross-platform benchmarking of the FHE libraries: novel insights into SEAL and OpenFHE. arXiv:2503.11216, 2025.
- [6] J. Takeshita, N. Koirala, C. McKechney, T. Jung. HEProfiler: an in-depth profiler of approximate homomorphic encryption libraries. *Journal of Cryptographic Engineering*, 2025.
- [7] A. Viand, P. Jattke, A. Hithnawi. SoK: fully homomorphic encryption compilers. *IEEE Symposium on Security and Privacy*, 2021.
- [8] M. Albrecht et al. Homomorphic Encryption Security Standard. HomomorphicEncryption.org, 2018.